Architecture Decision Record (ADR-001)

Migración de SQLite3 a PostgreSQL

Jeremy Hidalgo
Santiago Ballestero
Kendall Chacón
Proyecto MiniGestor

Octubre 2026

1. Contexto

Durante una sesión de trabajo realizada vía Zoom entre los integrantes del grupo de desarrollo, se reevaluó la infraestructura de persistencia de datos para el proyecto **MiniGestor**.

Originalmente, se había seleccionado **SQLite3** por defecto debido a su rápida configuración e integración nativa con Django, ideal para las fases iniciales de prototipado. Sin embargo, tras analizar los requerimientos no funcionales de concurrencia, escalabilidad y la necesidad de preparar el sistema para un entorno de producción real, se discutió la viabilidad de migrar a un sistema gestor de bases de datos relacional cliente-servidor más robusto: **PostgreSQL**.

2. Alternativas Evaluadas

- **SQLite3 (Opción Inicial):** Base de datos basada en archivos locales. Excelente para desarrollo offline y pruebas unitarias rápidas, pero con limitaciones significativas en concurrencia de escritura y características empresariales avanzadas.
- **PostgreSQL (Opción Seleccionada):** Base de datos relacional robusta, altamente concurrente, con soporte nativo avanzado para restricciones, tipos de datos complejos y una excelente compatibilidad con Django a través del conector `psycopg2`.

3. Decisión

Se toma la decisión unánime de **cambiar de SQLite3 a PostgreSQL** como el Sistema Gestor de Bases de Datos (SGBD) oficial para el proyecto MiniGestor, incorporándolo desde las fases de desarrollo y asegurando su estructura relacional desde el diseño actual.

4. Justificación

Las razones principales que motivaron este cambio son:

1. **Concurrencia y Rendimiento:** Un minisúper maneja operaciones frecuentes de caja y consulta de inventario en paralelo. PostgreSQL gestiona el control de concurrencia mediante bloqueos a nivel de fila de manera superior a SQLite3, evitando bloqueos de base de datos ante escrituras simultáneas.

2. **Estándar de Producción:** Utilizar PostgreSQL alinea el proyecto con arquitecturas empresariales reales, facilitando el despliegue en contenedores (Docker) y servicios cloud sin requerir refactorizaciones drásticas de la capa de datos.
3. **Integración con Django:** Django soporta PostgreSQL de forma nativa e integral a través del backend de base de datos correspondiente, permitiendo aprovechar al máximo las migraciones y el ORM.

5. Consecuencias

- **Positivas:** Mayor robustez del sistema, integridad referencial estricta, mejor rendimiento ante accesos concurrentes de cajeros y administradores, y cumplimiento de buenas prácticas arquitectónicas.
- **Negativas / Mitigación:** Se añade una pequeña dependencia inicial al requerir la instalación del servicio de PostgreSQL o el uso de Docker. Esto se mitiga documentando claramente los pasos de instalación en el repositorio Git del equipo.